

Introduction to Version Control

Problems Without Version Control

- No history of changes
- Difficult collaboration
- Risk of losing work
- Confusion with file versions

Real-World Examples

- Multiple document versions
- Team collaboration issues
- Breaking working code
- No way to revert mistakes

Benefits of Version Control

- Complete history tracking
- Easy collaboration
- Safe experimentation
- Time travel to any version

History of Git

1

2005 - The Problem

Linux kernel development relied on the proprietary version control system **BitKeeper**, but in 2005 the free license for Linux developers was revoked after a licensing dispute, forcing the project to find an alternative.

2

April 2005 - Creation

In April 2005, **Linus Torvalds** created **Git** to manage Linux kernel development. The first working version was built extremely quickly within about **10 days**, and continued evolving over the following months.

3

2008-Present - Growth

Since the late 2000s, Git has become the **most widely used version control system**, powering millions of projects and platforms like **GitHub** and **GitLab**.

Git was created because the Linux team suddenly lost their version control tool.

Git Fundamentals



Repository

A repository (repo) is a folder that stores your project files and the complete history of all changes.



Branch

A separate version of the project used to develop features without affecting the main code.



Remote Repository

A copy of your repository stored on a server (e.g., GitHub) for collaboration and backup.



Commit

A saved snapshot of your project at a specific point in time with a message describing the changes.



Merge

Combines changes from one branch into another (usually into the main branch).

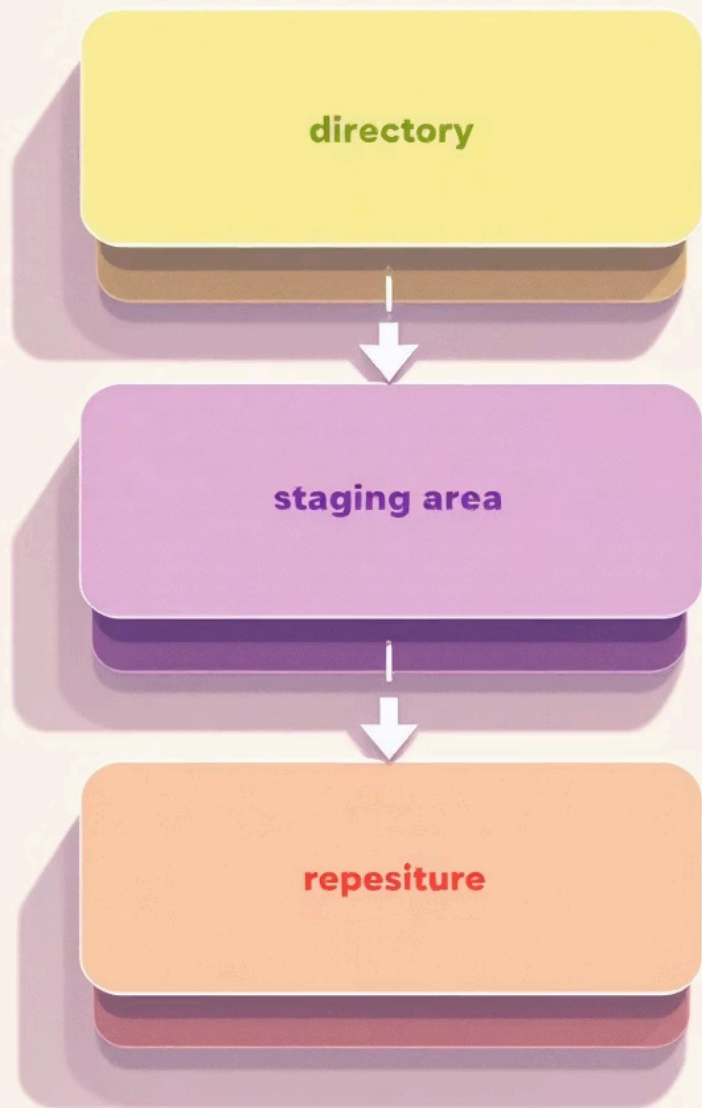


Local vs Remote

Local: repository on your computer.

Remote: repository stored online.

Use **push** and **pull** to synchronize them.

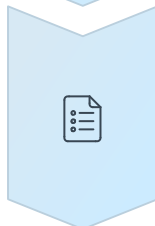


How Git Works: The Three Areas



Working Directory

The project files on your computer that you can create, edit, or delete.



Staging Area

A place where you **select and prepare changes** before creating a commit.



Git Repository

The database that stores all commits and the complete project history.

Installing Git

01

Download Git

Visit git-scm.com and download the appropriate installer for your operating system

03

Verify Installation

Open terminal and run `git --version` to confirm Git is installed.

If Git is installed correctly, it will show the installed version.

Git uses this information to label your commits so collaborators know who made each change.

02

Run Installer

Follow the installation wizard, keeping default settings for most options

04

Initial Configuration

```
git config --global user.name "Arun "
```

```
git config --global user.email "arun@neupane.com"
```

Check configuration:

```
git config --list
```

Essential Git Commands



git init

Initializes a new Git repository by creating a hidden `.git` folder that tracks project history.

```
git init
# Creates .git folder for tracking
```



git clone

Downloads a remote repository and creates a local working copy.

```
git clone https://github.com/user/repo.git
# Downloads entire project
```



git status

Shows the current state of your working directory and staging area

```
git status
# Lists modified, staged, and untracked files
```



git add

Adds changes to the staging area for the next commit

```
git add file.txt
git add .
# stages all changes in the current directory, including: new files, modified files, deletions
```



git commit

Saves a snapshot of staged changes with a descriptive message

```
git commit -m "Add new feature"
# Creates permanent record of changes
# Commits are stored in the repository history.
```



git log

Displays the commit history with messages, authors, and dates

```
git log
# Shows all commits from newest to oldest
git log --online
```

More Git Commands

git diff

Shows the differences between file versions that have not yet been committed.



```
git diff
# Displays line-by-line changes
```

git branch

Lists, creates, or deletes branches in your repository



```
git branch -> list branches
git branch new-feature -> create new branch with name new-feature
git branch -d new-feature -> Deletes a branch
```

git checkout / switch

git checkout (older command) and git switch (newer command) are used to change branches.



```
git checkout main
git switch main
# Changes to main branch
```

git merge

Combines changes from one branch into the current branch



```
git checkout main
git merge new-feature
# Merges feature into main
```

git pull

Downloads changes from remote and merges them into your local branch



git pull = **git fetch** + **git merge**

```
git pull origin main
# Fetches and merges remote changes
```

git push

Uploads your local commits to the remote repository



```
git push origin main
# Sends commits to remote
```

Typical Git Workflow





Collaboration with Git



Remote Repositories

Central locations like GitHub, GitLab, or Bitbucket store your code in the cloud



git pull

Always pull latest changes before starting work to avoid conflicts



Merge Conflicts

When changes overlap, Git flags conflicts requiring manual resolution

Hands-On Exercises

Task 1: Initialize and Commit

Practice the basic Git workflow with these steps:

1. Create a new folder and initialize Git
2. Add a text file with some content
3. Stage the file and commit with a message
4. Verify your commit with git log

```
mkdir my-project  
cd my-project  
git init  
echo "Hello World" > README.txt  
git add README.txt  
git commit -m "Add README"
```

Task 2: Branch and Merge

Practice branching workflow with these steps:

1. Create a new feature branch
2. Modify the README file in the branch
3. Commit your changes
4. Switch back to main and merge the branch

```
git branch new-feature  
git checkout new-feature  
echo "New feature" >> README.txt  
git commit -am "Add feature to README"  
git checkout main  
git merge new-feature
```